

Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	2 Marks

IDEATION PHASE

Define the Problem Statements

Customer Problem Statement Template: Create a problem statement to understand your customer's point of view. A well-articulated customer problem statement allows you and your team to find the ideal solution for the challenges your customers face. Throughout the process, you'll also be able to empathize with your customers, helping you understand how they perceive the platform security controls.

I am
Customer with 3-4 key characteristics – who are they?

IT Security Lead & System Administrator
managing enterprise records

I'm trying to
Outcome or job they care about achieving

Enforce granular, zero-trust record access based on field values

but
Problems or barriers that get in the way

Standard role ACLs permit blanket table access regardless of sensitivity

because
Root cause of why barrier exists – what to solve

Built-in condition builders cannot dynamically evaluate live field states

makes me feel
Emotions resulting from experiencing the barriers

Anxious, Exposed & Distrustful of data privacy

Customer Problem Statements Formulation Table

ID	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Security Operations Administrator in an enterprise IT service organization	Restrict visibility of security incident tickets where <code>severity = 'P1'</code> and <code>confidential = true</code>	All technicians holding the standard <code>itil</code> role can view sensitive breach data across the entire table	Default table-level ACL rules only evaluate static user roles rather than evaluating real-time field attributes	Vulnerable & stressed during audits
PS-2	HR Records Manager handling employee grievance and compensation records	Ensure that grievance records are restricted only to the assigned investigator and case manager	Support staff can run global search queries and view private employee disciplinary notes	The platform lacks a script-driven evaluation gate to cross-check current user ID with record field values	Compromised & distrustful of system
PS-3	Compliance & Risk Auditor validating SOC 2 and GDPR record segregation	Prevent unauthorized modification once records enter terminal states (<code>state = 'Closed'</code>)	General users with write privileges can still modify archived notes and historical financial impact fields	Condition builders cannot enforce complex business logic like immutable locking without custom script control	Non-Compliant & frustrated by breaches

Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	4 Marks

IDEATION PHASE

Empathize & Discover – Empathy Map Canvas

An empathy map is a visual synthesis capturing knowledge about our target persona: **Enterprise IT Administrator & Security Custodian**. It helps the engineering team examine the operational pains, systemic blockers, and access governance challenges from the user's direct perspective.

 **What does the user THINK & FEEL?**

"Coarse role-based permissions are leaking confidential ticket records!"

"We desperately need real-time script evaluation on `current.confidential`."

"Worried about failing quarterly GDPR, HIPAA, and ISO 27001 data isolation audits."

 **What does the user SEE?**

Over-privileged Tier-1 agents reading VIP customer complaints.

Unrestricted table lists returning records that have `confidentiality = High`.

Security audit dashboards flashing red due to unsegregated field access.

 **What does the user HEAR?**

CISO: "Why can any technician read executive payroll adjustment tickets?"

Helpdesk Manager: "We need automated field-based locks, not manual ticket reassignments."

Legal Compliance: "Every single privileged field view must be strictly verifiable."

 **What does the user SAY & DO?**

Writes ad-hoc Business Rules that conflict with standard platform access rules.

Advocates for server-side scripted ACLs evaluating `answer = (current.field == x)`.

Configures role validation combined with record ownership checks in JavaScript.

 **PAIN (Fears, Frustrations & Obstacles)**

Data Breaches: Accidental exposure of PII and salary figures via default table queries.

Clunky Workarounds: Splitting data into separate custom tables just to achieve simple privacy.

Audit Penalties: Financial fines and credential revocations from data governance lapses.

 **GAIN (Wants, Needs & Success Measures)**

Granular Access: Dynamic script execution evaluating field values before granting read/write.

Deterministic Security: Zero-trust record isolation operating seamlessly at server level.

Regulatory Compliance: Automatic compliance verification with tamper-proof audit trails.



Script-Controlled ACL – Restrict Record Access Based on Field Value

Ideation Phase • Page 2

Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	4 Marks

IDEATION PHASE

Brainstorm & Idea Prioritization Template

How Might We (HMW) Focus Statement: How might we dynamically restrict user read/write access to confidential records based on real-time field values (such as sensitivity tag, state, or department) while ensuring seamless system performance and zero unauthorized data leakage?

Step-1 & Step-2: Brainstorm, Idea Listing & Grouping

Team Member 1 (Security Architect)

Idea-1: Script-Controlled Read ACL

Server-side ACL script evaluating `current.confidentiality == 'Restricted'`. Returns `answer = true` only if the user possesses the `sec_analyst` role.

Team Member 2 (Platform Developer)

Idea-2: Before-Query Business Rule

Intercept list queries and automatically append an encoded filter (e.g., `u_restricted=false^ORassigned_to=gs.getUserID()`) to sanitize record listings.

Team Member 3 (UI/UX Specialist)

Idea-3: Client-Side UI & Data Policies

Enforce read-only state and mask fields on the client browser using UI Policies when a dropdown value changes to 'Closed' or 'Sensitive'.

Team Member 4 (ServiceNow Specialist)

Idea-4: Hybrid Scripted ACL & Ownership Gate

Combine field value checks with record assignment checks: permits access only if user is Case Owner OR member of the Assigned Support Group.

Team Member 5 (Audit Analyst)

Idea-5: Automated Security Audit Logger

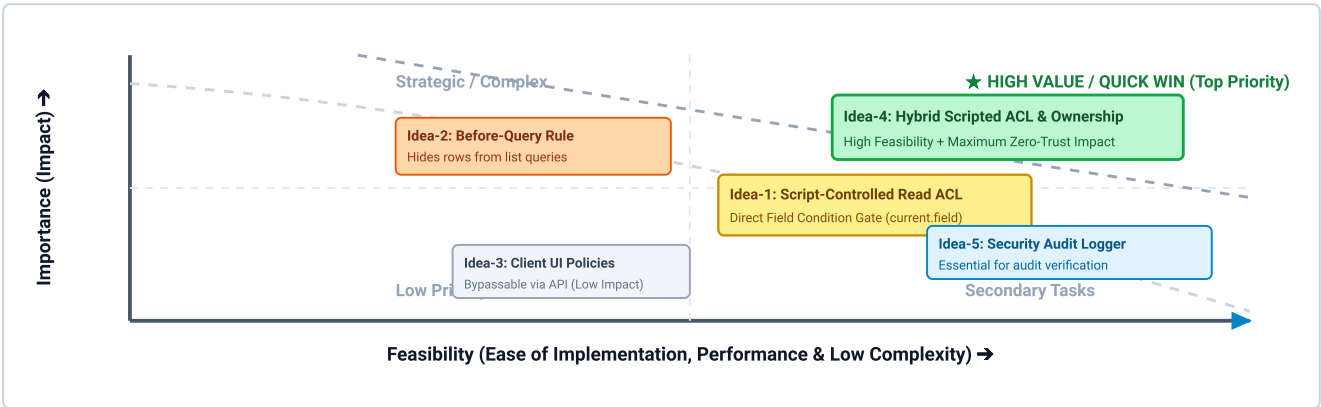
Trigger automated system event logs whenever a script-controlled ACL evaluates to false, logging the user ID, timestamp, and accessed record number.

Team Member 6 (Systems Integrator)

Idea-6: Role Hierarchy & Field Inheritance

Implement dynamic role inheritance where sensitive field flags automatically prompt for an elevated one-time security token before unlocking record rows.

Step-3: Idea Prioritization (Importance vs. Feasibility Matrix)



Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	4 Marks

PROJECT DESIGN PHASE-II

Solution Requirements (Functional & Non-Functional)

Functional Requirements:

Following are the functional requirements (Epics and Stories) designed for the script-controlled access governance engine.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Dynamic Field-Value Read Restriction	- Evaluate <code>current.u_confidentiality_level == 'Restricted'</code> on record load. - Enforce read denial unless user has <code>security_admin</code> or is the designated record owner. - Mask sensitive field data when displayed in multi-row table list views.
FR-2	State-Based Write Lockout Control	- Intercept record modification requests when <code>current.state == 'Closed' or 'Archived'</code>. - Script evaluates <code>answer = false</code> to lock entire record row from unprivileged edits. - Permit field modifications only for authorized Compliance Officers with audit logging.
FR-3	Contextual Record Ownership Gate	- Server-side script validation: <code>current.assigned_to == gs.getUserID()</code>. - Allow department-level view privileges if <code>user.department == current.caller_id.department</code>. - Dynamic group-membership check via <code>gs.getUser().isMemberOf(current.assignment_group)</code>.
FR-4	Security Violation Audit Trail	- Generate an immutable entry in <code>sys_audit</code> whenever an ACL script blocks an access attempt. - Capture originating client IP address, transaction timestamp, user ID, and target record <code>sys_id</code>.

Non-Functional Requirements:

Following are the non-functional quality attributes governing reliability, performance, and compliance.

NFR No.	Non-Functional Requirement	Description
NFR-1	Security	Server-side sandbox enforcement preventing client-side script tampering or API bypassing; strict zero-trust posture with fail-secure defaults.
NFR-2	Performance	Script execution latency must not exceed 15ms per record evaluation. ACL queries must avoid nested GlideRecord queries to preserve throughput.
NFR-3	Reliability	99.99% deterministic access outcome; if runtime script exceptions occur, system must fail securely by denying access and alerting admins.
NFR-4	Usability	Provide user-friendly field-level security annotations ("Security Restricted") rather than uninformative page crash errors or broken UI states.
NFR-5	Availability	Continuous 24/7 rule enforcement without table locks or latency degradation during ACL script updates and security cache flushing.
NFR-6	Scalability	Architecture must effortlessly support 100,000+ daily record evaluations and concurrent query operations across enterprise service desks.

Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	4 Marks

PROJECT DESIGN PHASE-II

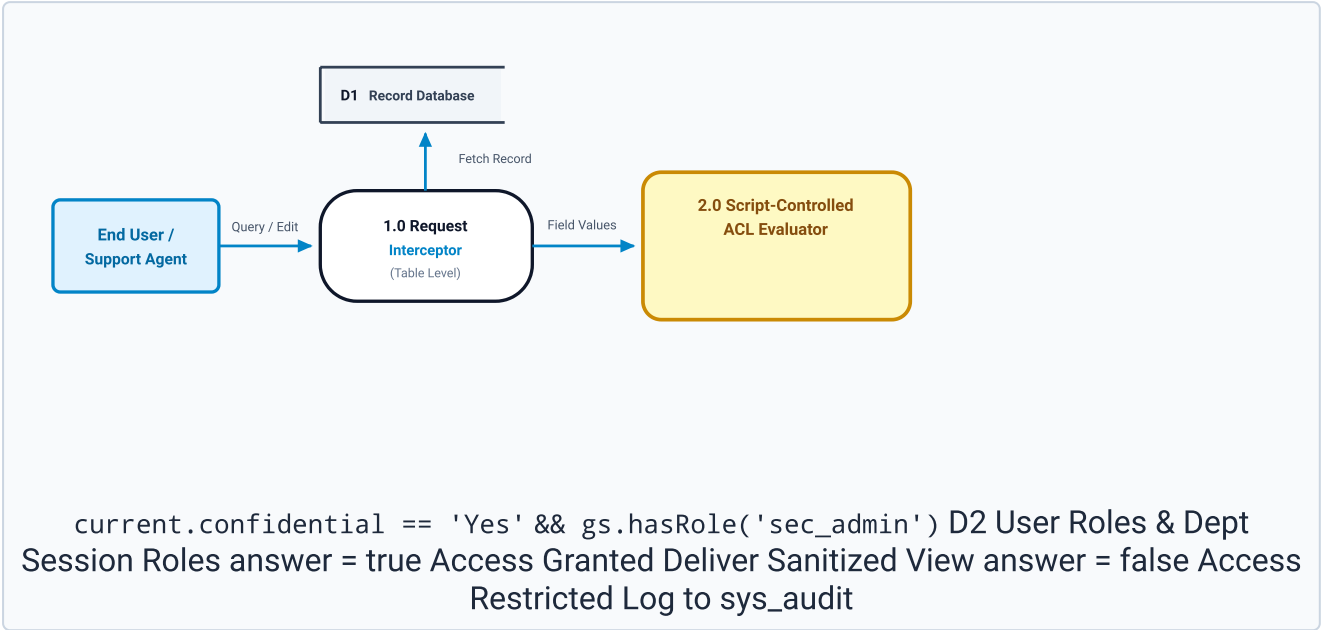
Data Flow Diagram & Process Architecture

Data Flow Diagrams (DFD): A Data Flow Diagram depicts how information enters, transits, and leaves the system. In this solution, it illustrates how incoming user record requests trigger server-side scripted evaluation against live field values and user session variables before granting access.

Execution Flow:

- 1. **Request Dispatch:** User or client application initiates a query (Read/Write) for a specific record in the table.
- 2. **Record Attribute Retrieval:** The persistence layer fetches the target record attributes (e.g., state, u_confidential, assignment_group).
- 3. **Script-Controlled ACL Trigger:** The security engine intercepts the transaction and invokes the server-side JavaScript evaluator.
- 4. **Contextual Rule Evaluation:** The script validates current.field_value against active user session tokens (gs.getUserID(), roles, group memberships).
- 5. **Access Arbitration & Logging:** The ACL engine sets answer = true to release data or answer = false to trigger audit logging.

Example: DFD Level 0 & Level 1 (Industry Standard Architecture)



Date	31 January 2025	Team ID	[Team ID]
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value	Maximum Marks	4 Marks

PROJECT DESIGN PHASE-II

User Stories Specification

Use the below template to list all the user stories, acceptance criteria, priority rankings, and target release sprints for the script-controlled ACL platform deployment.

User Stories & Acceptance Criteria:

User Type	Epic	Story ID	User Story / Task	Acceptance Criteria	Priority	Release
Tier-1 Support Agent	Field-Level Read Restriction	USN-1	As a tier-1 agent, I should not be able to view tickets marked <code>confidential = true</code> , so sensitive customer information is safeguarded.	Record is omitted from list queries; direct URL access displays "Access Denied by Security Policy".	High	Sprint-1
Case Owner	Dynamic Access Evaluation	USN-2	As a case owner, I can access and modify my own assigned records even if marked sensitive, to continue resolving the issue.	Script checks <code>current.assigned_to == gs.getUserID()</code> and returns <code>answer = true</code> .	High	Sprint-1
Department Manager	Department Segregation	USN-3	As a department manager, I want to view confidential records created within my department to manage departmental reporting.	Script verifies <code>user.department == current.caller_id.department</code> ; unlocks read view for verified manager.	Medium	Sprint-1
Security Administrator	Privileged Override	USN-4	As a security admin, I need override access to any locked record for emergency investigation with mandatory rationale entry.	User with <code>security_admin</code> role can bypass field restriction; system triggers audit record in <code>sys_audit</code> .	High	Sprint-2
Compliance Officer	State-Based Record Lockdown	USN-5	As a compliance officer, I want write access blocked for all users when <code>state == 'Closed'</code> , to prevent retroactive record tampering.	Write ACL script checks <code>current.state == 'Closed'</code> and forces read-only view on form and list editor.	High	Sprint-2
System Auditor	Audit & Compliance Alerting	USN-6	As an auditor, I want unauthorized attempts to access confidential fields to be logged with timestamps and user identifiers.	Security dashboard reports real-time ACL evaluation rejections, highlighting user ID, client IP, and target <code>sys_id</code> .	Medium	Sprint-2